CHROMADB IN AI MEMORY BUS

How the Semantic Memory Layer Works

Project-specific usage guide — brief

all-MiniLM-L6-v2 · 384-dim · cosine · HNSW

Built by Bhavesh

1. What ChromaDB Does in This Project

In AI Memory Bus, ChromaDB is the **semantic memory store** — one of four memory types (working, episodic, semantic, procedural). It stores facts about the user as vector embeddings so any of the six connected AI providers (Claude, GPT-4, Gemini, Groq, Cohere, Ollama) can recall them by meaning, not exact wording.

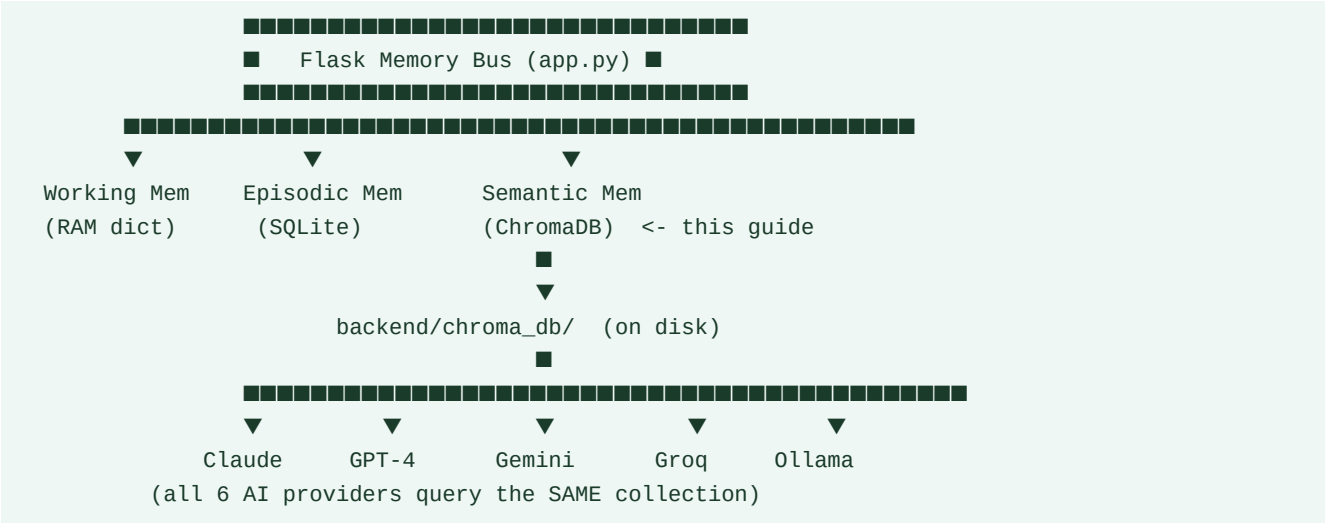
> **In this project:** there is no multi-agent bus and no agent_id metadata. There is one user, one shared collection, and many AI providers reading from the same memory. Whatever Claude learns about you today, GPT-4 can recall tomorrow — because both query the identical ChromaDB collection.

Why ChromaDB specifically (not Postgres / Redis)

Recall by meaning	"What does the user prefer?" matches "User likes Python" even with zero shared keywords.
No server needed	PersistentClient writes straight to backend/chroma_db/ — no separate database process to run.
Built-in HNSW	Approximate nearest-neighbour search ships out of the box — no separate index to build.
Confidence pairs naturally	Metadata stores confidence and reinforcement count alongside each vector for decay scoring.

2. Where It Sits in the Architecture

ChromaDB is one of three storage engines behind the Memory Bus. Working memory (RAM) and episodic memory (SQLite) sit beside it; ChromaDB is reserved for long-term semantic facts.



How it's reached in the request flow

From the request-flow pipeline: Step 2 (Fetch Memories) queries this collection for the top-5 semantically similar facts, Step 3 (Safety Net) re-queries it even when routing rules didn't select "semantic", and Step 6 has no write-back here — new semantic facts are written separately when the system extracts a durable preference from conversation.

3. Setup Used in This Project

Client mode	PersistentClient(path="backend/chroma_db") — single Flask process, single machine, local-first by design.
Embedding model	all-MiniLM-L6-v2 via sentence-transformers — 384 dimensions, runs offline, no API key, ~5-15ms per embedding on CPU.
Distance space	cosine — set once via metadata={"hnsw:space": "cosine"} at collection creation. Cannot change later without recreating it.
Collection	One collection for all semantic facts about the user. No per-provider isolation — that would defeat cross-AI sharing.

```
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="backend/chroma_db")

embed_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

semantic_memory = client.get_or_create_collection(
    name="semantic_memory",
    embedding_function=embed_fn,
    metadata={"hnsw:space": "cosine"},
)
```

4. Metadata Schema Used Here

No agent_id field — there's one user, not multiple agents. Metadata instead tracks confidence and decay, which is specific to this project's memory model.

Field	Type	Purpose
content	string	The fact text, e.g. "User prefers Python and FastAPI"
confidence	number	0.0–1.0, decays over time; gate for injection (>= 0.70)
reinforcement	number	Times this fact was restated; slows confidence decay
source	string	Which AI/session the fact was first learned from
ts	number	Epoch seconds — used to compute decay, not for filtering
tags	string	Comma-joined labels, e.g. "preference,backend"

Write path

```
def remember_fact(content, confidence=0.9, source="conversation", tags=None):
    semantic_memory.add(
        ids=[str(uuid.uuid4())],
        documents=[content],
        metadatas=[{
            "confidence": confidence,
            "reinforcement": 1,
            "source": source,
            "ts": time.time(),
            "tags": ",".join(tags or []),
        }],
    )
```

Read path — used in Step 2 of the request flow

```
def recall_facts(query, n=5):
    res = semantic_memory.query(query_texts=[query], n_results=n)
    results = list(zip(res["documents"][0], res["metadatas"][0], res["distances"][0]))
    # Apply this project's confidence + similarity filter:
    return [
        (doc, meta) for doc, meta, dist in results
        if meta["confidence"] >= 0.70 and (1 - dist) >= 0.75
    ][:3]   # hard cap: 3 semantic memories per request
```

5. Why the Filtering Rules Exist

similarity >= 0.75	Below this, ChromaDB's nearest match is often unrelated — injecting it would mislead the AI rather than help it.
confidence >= 0.70	A fact can be semantically on-target but stale (decayed). This is a project-level filter, not a ChromaDB feature — confidence lives in metadata, ChromaDB only returns distance.
Max 3 semantic results	Keeps total injected context (across all memory types) under 5 — avoids context dilution in the AI's system prompt.
Safety-net re-query	Even if keyword routing didn't flag "semantic", a query still runs; results scoring similarity >= 0.80 get injected anyway (max 2) — catches relevant facts routing missed.

6. Limitations That Matter Here

Single-writer in embedded mode	PersistentClient is single-process. Fine for one local Flask server; would need server mode (HttpClient) only if scaling to multiple processes.
No transactions	ChromaDB isn't where current_task or active_file lives — that's working memory (RAM) precisely because it needs none of ChromaDB's strengths.
Embedding model is fixed	Switching from all-MiniLM-L6-v2 later means re-embedding every stored fact — old and new vectors aren't comparable.
Confidence isn't built-in	ChromaDB has no decay concept — confidence decay is computed in this project's code, stored as plain metadata, and read back on every query.

7. Quick Reference

```
client = chromadb.PersistentClient(path="backend/chroma_db")     # connect
col    = client.get_or_create_collection("semantic_memory")       # get store
col.add(ids=[id], documents=[text], metadatas=[{...}])             # write fact
col.query(query_texts=[q], n_results=5)                            # semantic read
col.upsert(ids=[id], documents=[...])                              # correct a fact
col.delete(ids=[id])                                               # forget a fact
col.count()                                                        # how many facts stored
```

One-line summary for this project: ChromaDB is the long-term "what is true about this user" store — one collection, cosine similarity, all-MiniLM-L6-v2 embeddings, filtered by similarity (>= 0.75) and confidence (>= 0.70), capped at 3 results per request — so every connected AI provider recalls the same facts without ever talking to each other directly.